

INTRODUCTION

- * Basically algorithm is a finite set of instructions that can be used to perform certain task.
- * In this chapter we will learn some basic concepts of algorithm.
- * We will start our discussion by understanding the notion of algorithm & how to write an algorithm & analyze the algorithm in the following section with some examples.

ALGORITHM

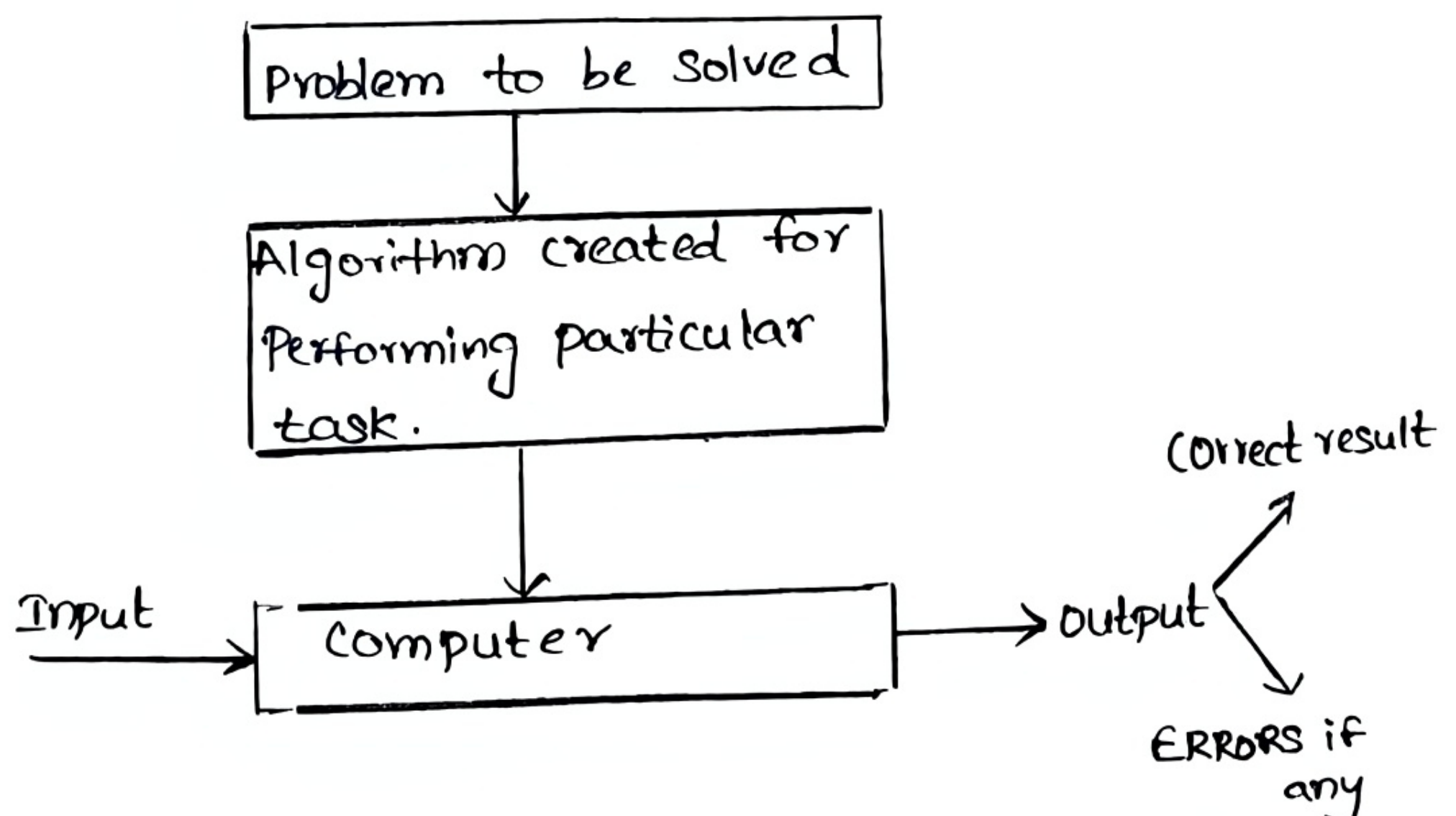
- * The algorithm is defined as a collection of unambiguous instructions occurring in some specific sequence and such an algorithm should produce output for given set of input in finite amount of time.
- * This definition of algorithm is represented in figure
- * After understanding the problem statement, we have to create an algorithm carefully for the given problem.

- * The algorithm is then converted into some programming language and then given to some computing device (computer)
- * The computer then executes this algorithm which is actually submitted in the form of Source program.
- * During the process of execution it requires certain set of input.
- * With the help of algorithm (in the form of program) and input set, the result is produced as an output.
- * If the given input is invalid then it should raise appropriate error message (or) otherwise correct result will be produced as an output.

Common terms used in Algorithm:

- 1) DATATYPES: Set of variables takes its values.
- 2) VARIABLES: It refers to specific location in computer memory used to store 1 & only 1 value.
- 3) STATEMENT: It just a line of codes is called statement.

FIG: NOTION OF ALGORITHM



PROPERTIES OF AN ALGORITHM / CHARACTERISTICS OF AN ALGORITHM: (2)

An algorithm must have the following properties:

1. Input : An algorithm use values from a specific set.
2. output : for each input of an algorithm produces values from a specific task
3. Correctness: It should produce the output according to the requirement of the algorithm.
4. finiteness : Algorithm must complete after a finite number of instructions have been executed.
5. Definiteness: All steps of the algorithm must be precisely defined.
6. feasibility : It must be feasible to execute each instruction
7. AN Absence of Ambiguity: Each step must be defined, having Only one Interpretation.
8. Efficient : Efficiency is always measured in terms of space and time requires implementing the algorithm, so the algorithm uses a little running time and memory space as possible within the limits of acceptable development time.
9. Generality : The algorithm applies to a set of inputs.

Note:

* Here so many properties are defined in algorithm. The main characteristics of a good algorithm are

1. Input 2. Output 3. Definiteness 4. finiteness 5. Generality
6. Correctness 7. Precision (The steps are precisely stated).

Example for an algorithm:

We need to sort the sequence of number into ascending order.

Here is how we define the sorting problem.

* Input :-

A sequence of n number $(a_1, a_2, \dots, a_n)$

* output:-

A permutation (reordering $(a'_1, a'_2, \dots, a'_n)$) of the input sequence such that $(a'_1 \leq a'_2 \leq \dots \leq a'_n)$

NEED OF ALGORITHM :

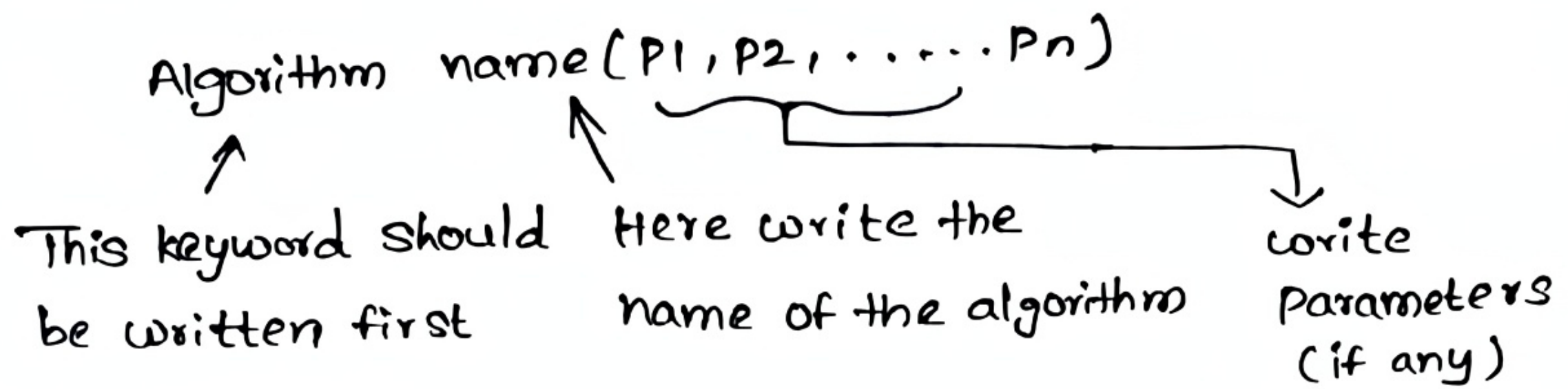
1. To understand the basic idea of the problem.
2. To find an approach to solve the problem.
3. To improve the efficiency of existing techniques.
4. To understand the basic principles of designing the algorithms.
5. To compare the performance of the algorithm w.r.t other techniques.
6. It is the best method of description without describing the implementation detail.
7. The Algorithm gives a clear description of requirements and goal of the problem to the designer.
8. A good design can produce a good solution.
9. To understand the flow of the problem.
10. To measure the behaviour (or performance) of the methods in all cases (best cases, worst cases, average cases)
11. To understand the principle of designing
12. we can measure and analyze the complexity (time & space) and it will reduce the cost of design.

PSEUDO CODE FOR EXPRESSING ALGORITHMS

(3)

* Let us understand some rules for writing the algorithm:-

1. Algorithm is a procedure consisting of heading and body.
The heading consists of keyword Algorithm and name of the algorithm and parameter list. The Syntax is



2. Then in the heading Section we should write following things:

// problem Description :

// input :

// output :

3. Then body of an algorithm is written, in which various Programming constructs like if, for, while (or) some assignment Statements may be written.
4. The compound statements should be enclosed within { and }
5. Single line comments are written using // as beginning of comment.
6. The identifier should begin by letter and not by digit. An Identifier can be a combination of alphanumeric string.
7. Using assignment operator $\leftarrow$ an assignment statement can be given.

for Instance:

Variable $\leftarrow$ expression

8. There are other types of operators such as Boolean operators such as true (or) false. Logical operators such as and, or, not. And relational operators such as $<$, $>$, $=$, $>=$, $<=$ & so..on
9. The array indices are stored with in square brackets '[]'. The index of array usually start at zero. The multidimensional array can also be used in algorithm.
10. The inputting and outputting can done using read and write.

for eg:

```
write ("This message will be displayed on console");  
read (val);
```

11. The Conditional Statements such as if-then (or) if-then-else are written in following form:

if (condition) then statement

if (condition) then statement else statement

→ If the if-then statement is of compound type then { and } should be used for enclosing block.

12. While statement can be written as:

```
while (condition) do
```

```
{
```

```
Statement 1
```

```
Statement 2
```

```
:
```

```
Statement n
```

```
}
```

→ while the condition is true the block enclosed with { } gets executed otherwise statement after } will be executed.

13. The general form for writing for loop is

④

for Variable $\leftarrow$ value₁ to value_n do

{

Statement 1

Statement 2

⋮

statement n

}

Here

Value₁ is initialization condition &

value_n is a terminating condition.

→ Sometime a keyword Step is used to denote increment
(or) decrement the value of variable for example

for i $\leftarrow$ 1 to n Step 1

{

write(i)

}

Here variable i is
incremented by 1
at each iteration

14. The repeat - until statement can be written as :

repeat

Statement 1

Statement 2

:

:

statement n

until (condition)

15. The break statement is used to exit from inner loop. The
return statement is used to return control from one point
to another. Generally used while exiting from function.

Some Examples :-

Example 1 :

Write an algorithm to count the sum of n numbers.

Sol: Algorithm Sum ($1, n$)

// problem Description: This algorithm is for finding the sum

// of given n numbers

// Input : 1 to n numbers

// output : The sum of n numbers

result $\leftarrow 0$

for $i \leftarrow 1$ to n do $i \leftarrow i + 1$

result $\leftarrow$ result + i

return result.

Example 2 :

Write an algorithm to check whether given number is even
(or) odd.

Sol: Algorithm eventest (val)

// problem Description: This algorithm test whether given
// number is even (or) odd.

// Input : The number to be tested i.e val

// output : Appropriate messages indicating even (or) oddness

if ($val \% 2 = 0$) then

write ("Given number is even")

else

write ("Give number is odd")

PROCESS OF DESIGN ANALYSIS AND ALGORITHM (5)

* Algorithm design:

A sequence of steps involved in designing and analyzing of an algorithm.

These steps are:-

1. understanding the problem
2. Decision making on
 - a) capabilities of computational devices.
 - b) choice for either exact (or) approximate problem solving method.
 - c) Data structures.
 - d) Algorithmic strategies
3. Specification of algorithm / Design an Algorithm
4. Algorithmic Verification / prove correctness
5. Analysis of algorithm
6. Implementation or coding of algorithm.

PERFORMANCE ANALYSIS

⑨

- * The efficiency of an algorithm can be decided by measuring the performance of an algorithm
- * We can measure the performance of an algorithm by computing two factors.
 1. Amount of time required by an algorithm to execute.
 2. Amount of storage required by an algorithm.
- * This is popularly known as time complexity and space complexity of an algorithm.

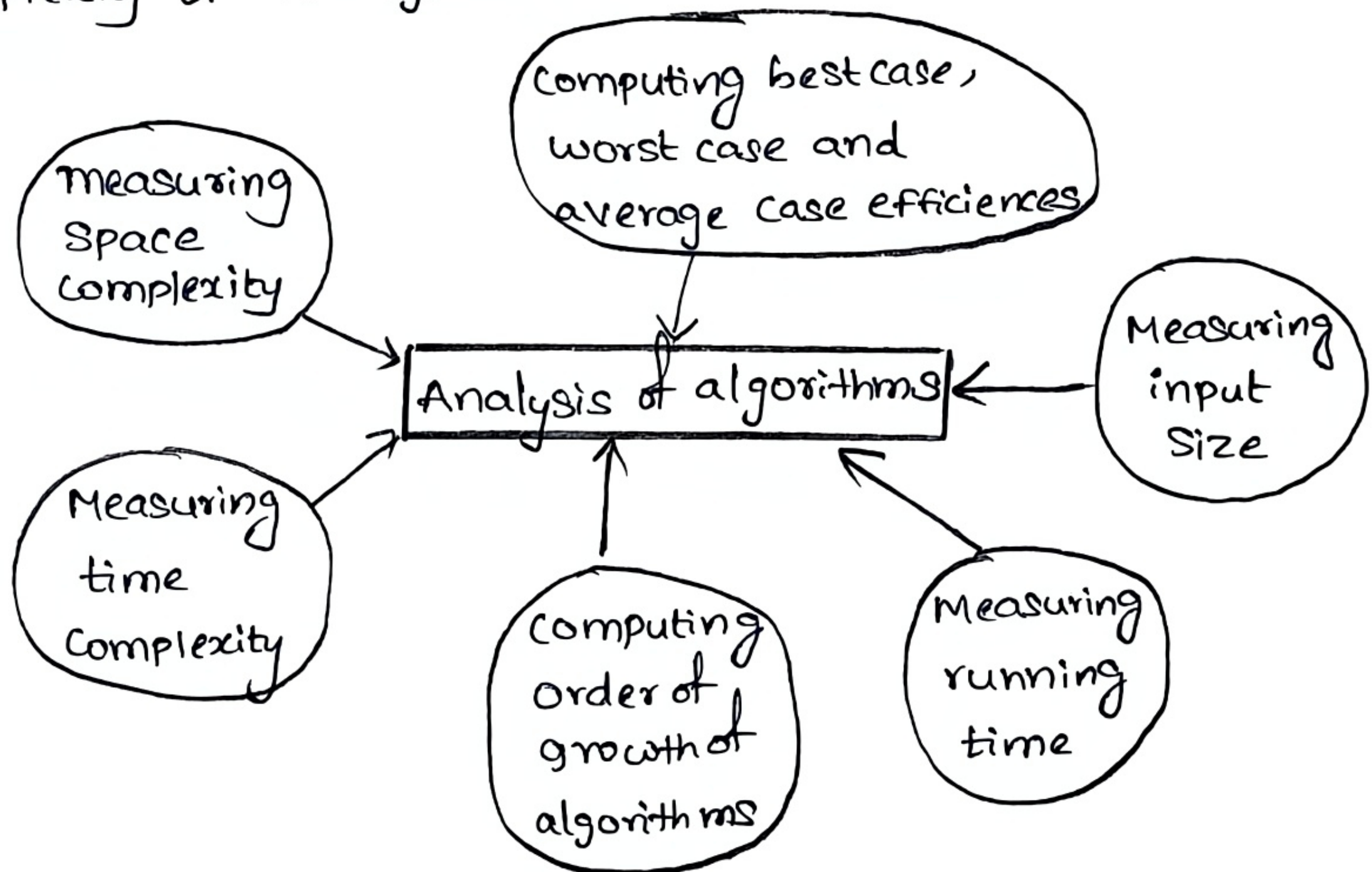


fig: Analysis of algorithms

1. SPACE COMPLEXITY:

The space complexity can be defined as amount of memory required by an algorithm to run

→ To compute the space complexity we use two factors

1. Constant / fixed part

2. Instance characteristics / Variable part.

→ The Space requirement $S(P)$ can be given as

$$S(P) = C + S_p$$

$S(P)$ → space requirement P → be the algorithm name.

C → constant i.e fixed part.

S_p → Variable part

FIXED PART :

→ It denotes the space of inputs and outputs

→ Includes space needed for storing instructions
Variables, constants and structured variables [arrays,
struct].

VARIABLE PART:

→ This variable part whose space requirement depends on particular problem instance

→ Includes space needed for stack & for structured variables that are dynamically allocated during run time

→ Typically include

1. Space needed by referenced variables

2. Recursion stack space & so. on

RULES:

* Size of variable like n, a, b assigns - 1 word

* Array values it assigns - n words

* In Loop variable assignment operators assigns - 1 word

Example 1:

(10)

Consider the following algorithm & calculate space complexity

1) write an algorithm for sum of n elements in the array & calculate space complexity.

Sol Algorithm Sum(a, n)

// problem Description: The algorithm for sum of n elements

// in the array

// Input: An array a & n is total number of elements

// in the array.

// Output: returns the value to the sum's and stored in that variable & display output.

{

$S \leftarrow 0$

for $i \leftarrow 1$ to n do

$S \leftarrow S + a[i]$

return S ;

}

The space requirement for the above algorithm is

$$S(P) = C + Sp$$

$$S(\text{Sum}) = C + Sp$$

$\therefore S \leftarrow 0$ is assigning the value so 1 word

$\therefore i = 1$ assignment operator so 1 word.

$\therefore n$ variable so 1 word.

$\therefore$ array a value $a[i]$ so n word.

Therefore

$$S(P) = (n+8) \text{ words}$$

for Recursive Algorithm

eg: Sum of n elements.

Sol Algorithm $\text{sum}(a, n)$

// Problem Description: This is a recursive algorithm

// which computes addition of all the elements in an array $a[]$

// Input: $a[i]$ is integer type, total number of elements in an array.

// Output: returns addition of ' n ' elements of an array

return ^{Sum} ~~Add~~ $(a, n-1) + a[n]$

$\therefore$ The Space requirement is

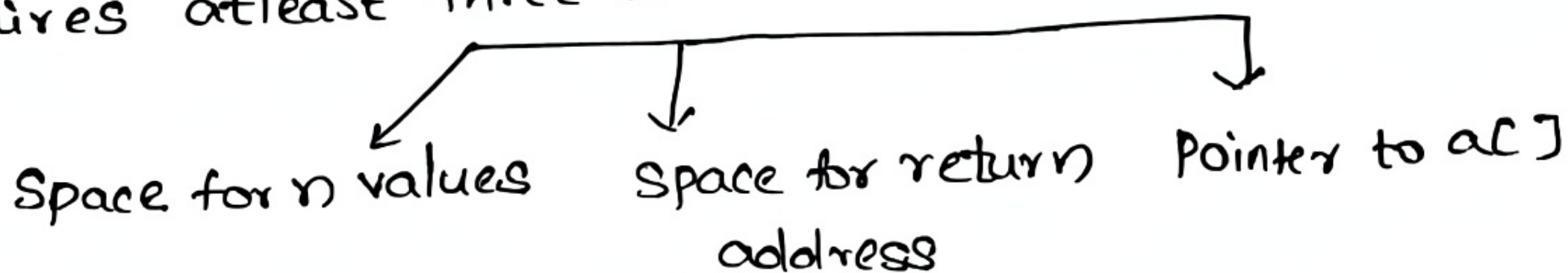
$$S(P) = 3(n+1)$$

Note:

→ The internal stack used for recursion includes space for formal parameters, local variables & return address.

→ The Space required by each call to function 'Sum'

requires at least three words



→ The depth of recursion is $n+1$ (n -times call to function & one return call). The recursion stack space will be $3(n+1)$ words.

2) TIME COMPLEXITY:

(ii)

- The time complexity of an algorithm is the total amount of time required by an algorithm to complete its execution
 - The time complexity is therefore given in terms of frequency count.
 - frequency count is a count denoting number of times of execution of statement.
 - Two components are used to determine the time complexity
 1. Compile time (or) constant time complexity
 2. Run time
1. Compile time :-
- It does not depend on instance characteristics
 - Once compiled program will be run several times without recompilation
- Constant time complexity:
- If any program requires fixed amount of time for all input values then its time complexity is said to be constant time complexity.
- eg:

```
int sum(int a, int b)
{
    return a+b;
}
```
2. Run time :-
- It is dependent on particular problem instance [dependent on 'I/p & O/p']

∴ Time complexity calculated in the form of

$$T(P) = C + T_P$$

where

$T(P)$ = Time complexity of Algorithm P

C = Compile time

T_p = Run time.

How to calculate Time complexity:-

1. calculate Time complexity will become easy by counting the no. of steps each statement in the program executes.
2. A program step is a segment of a program that is Independent of Instance characteristics & execution on problem statements
3. Identify the executable statements in the program and count the number of steps in the program assigned.

Rules:

Comments : 0 steps

for Assignment Statement : 1 step

Condition Statement : 1 step

Loop condition for 'n' times : (n+1) steps

Body of loop : n steps

* Based on this rules, we are going to calculate the time complexity for any algorithm.

Example: calculate the time complexity for sum of n elements in the array.

Sol: Statement no : No. of lines

Statement : program of sum of n elements written in pseudocode.

* S/E : Status of execution

(12)

* frequency count : The time complexity is measured in the terms of a unit called frequency count. The time which is measured for analyzing an algorithm is generally running time.

* Total steps : Multiply both S/E & frequency & place it in Total steps table.

* finally add all the Total steps for the algorithm & you will get time complexity for an algorithm.

Stmt no	Statement	S/E	frequency	Total steps
1.	Algorithm sum(a, n)	0	-	0
2.	{	0	-	0
3.	$S \leftarrow 0$	1	1	1
4.	for $i \leftarrow 1$ to n do	1	$(n+1)$	$(n+1)$
5.	$S \leftarrow S + a[i]$	1	n	n
6.	return S	1	1	1
7.	}	0	-	0
Total Steps for Algorithm :				$1 + (n+1) + n + 1$

$S \leftarrow 0$: Assignment operator so 1 step

for $i \leftarrow 1$ to n do : Loop statement iteration is performing so $(n+1)$ steps

$S \leftarrow S + a[i]$: body of the loop 'n' times

return S : for return statement $\rightarrow$ 1 step

$\therefore$ Time Complexity for the algorithm is

$$T(P) = (2n + 3) \text{ steps.}$$

Ex 2: calculate ^{TC for} two matrix for the algorithm?

Sol

Stmt no	Statement	S/E	frequency	Total steps
1.	Algorithm Add (a, b, c, m, n)	0	0	0
2.	{	0	0	0
3.	for i ← 1 to m do	1	(m+1)	(m+1)
4.	for j ← 1 to n do	1	m(n+1)	(mn+m)
5.	c[i][j] ← a[i][j] + b[i][j]	1	mn	mn
6.	}	0		
Total steps for Algorithm :				(m+1) + (mn+m) + mn

∴ Time complexity for the algorithm is

$$T(P) = (m+1) + (mn+m) + mn$$

$$= m+1 + mn+m+mn$$

$$T(P) = 2m + 2mn + 1 \text{ steps}$$

) ORDER OF GROWTH:

- Order of growth provide the behaviour of an algorithm.
- It gives a Simple characterization of the algorithm's efficiency.
- It allows us to compare the relative performance of alternate algorithms.
- It is done based on the primitive operations like arithmetic, relational, logical operations.
- Order of growth is defined when the i/p size of an algorithm increases then the computational time of an algorithm also increases.
i.e input is directly proportional to computational time.
- The running time of an algorithm depends on input size 'n' and nature of input.
- Order of growth depends on the efficiency of ^{cases} ~~cases~~ that is (Best-case, Average case, worst case)

→ order of growth follows the relation below .

(14)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < \dots$$

$$\dots < O(2^n) < O(n!)$$

→ In order of growth two bounds are used for analyzing the functions

1. upper bound

2. Lower bound

→ For example: If we want to identify the upper bound and lower bound function of $O(n)$.

Then

* Lower bound defines the below the $O(n)$ function growth

* upper bound defines the after the function growth .

* Here

Lower bound of $O(n)$ is : $O(1)$, $O(\log n)$

upper bound of $O(n)$ is : $O(n \log n)$, $O(n^2)$, $O(n^3)$,
 $O(2^n)$, $O(n!)$

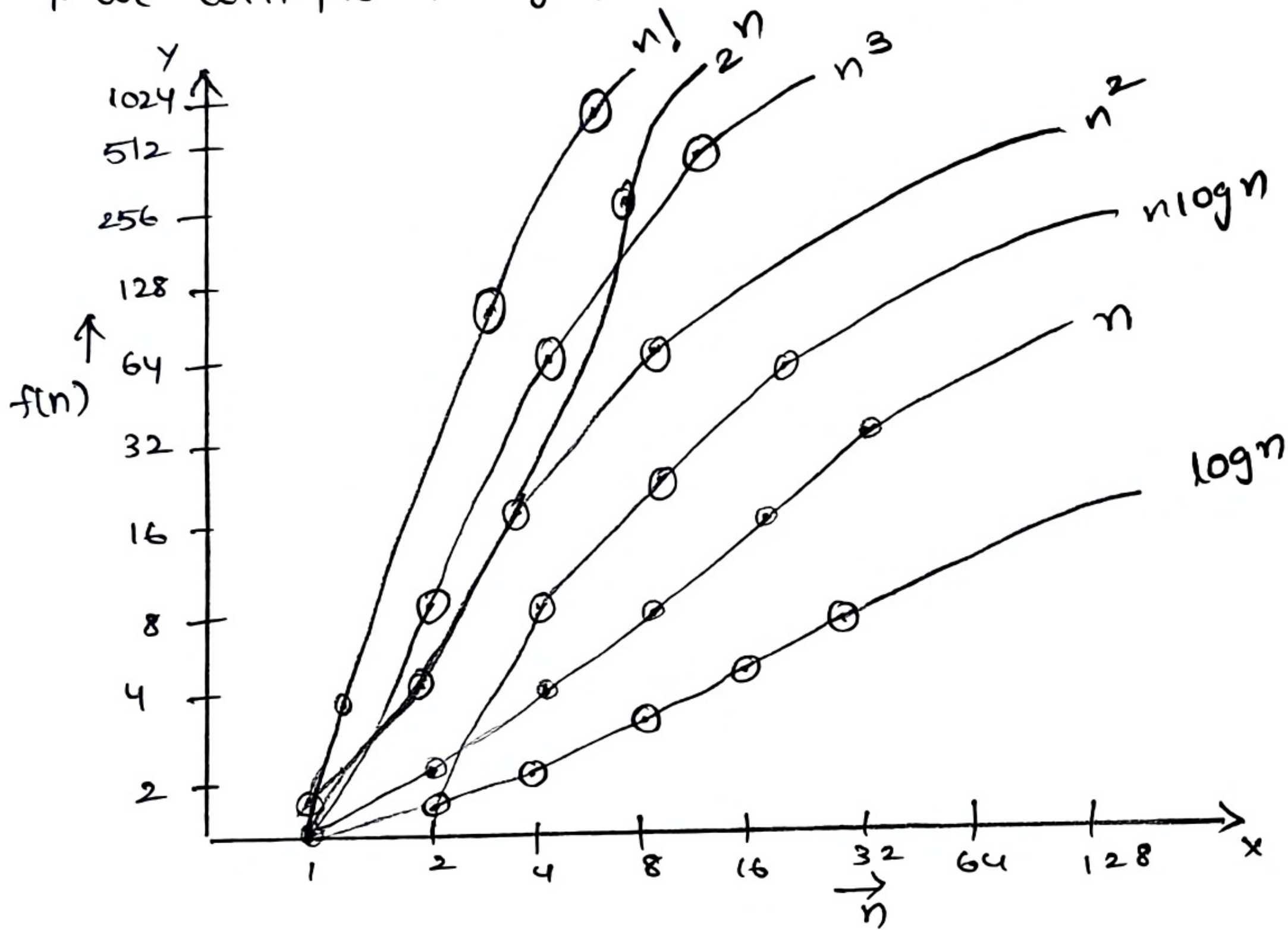
Example Table:

* Measuring the performance of an algorithm in relation with the input size n is called order of growth.

* for eg: the order of growth for varying input size of 'n' is as given below.

$f(n)$	n	$\log n$	$n \log n$	n^2	n^3	2^n	$n!$
n	1	0	0	1	1	2	1
	2	1	2	4	8	4	2
	4	2	8	16	64	16	24
	8	3	24	64	512	256	10080
	16	4	64	256	4096	65536	...
	32	5	160	1024	32768	4294967	...

* we will plot the graph for these values.



$n \rightarrow$ i/p size

$f(n) \rightarrow$ growth of function according to the i/p size.

fig: Rate of growth of common computing time function

* from the above graph it is clear that the logarithmic function is the slowest growing function & 2^n is fastest and grows rapidly with varying input size n .

* Here order of growth function $f(n)$ is defined with notations (15)

$O(1)$: constant

$O(\log n)$: logarithmic

$O(n)$: Linear

$O(n \log n)$: logarithmic

$O(n^2)$: quadratic

$O(n^3)$: cubic

$O(2^n)$: Exponential

$O(n!)$: factorial.

7. COMPUTING BEST CASE, WORST CASE & AVERAGE CASE EFFICIENCIES:

- 1) BEST CASE: It is the minimum number of steps that can be executed for a given parameter.
- 2) WORST CASE: It is the maximum number of steps that can be executed for a given parameter.
- 3) AVERAGE CASE: It is the average number of steps executed for a given parameter.

Examples:

Eg 1) Linear Search (Sequential Search)

1) BEST CASE:

* If we want to search an element 2, whether it is present in the array (or) not.

	2	4	5	6	8	9	10	11	13
A	1	2	3	4	5	6	7	8	9

- * first $A(1)$ is compared with 2, Match occurs. So the number of comparisons is only one
- * It is observed that Search takes minimum no. of comparisons, So it come under best case.

$\therefore$ Time complexity is $O(1)$

AVERAGE CASE:

- * If we want to Search an element 8, whether it is present in the array (or) not.
- * first, $A(1)$ is compared with 8 i.e $(2=8)$, no match occurs.
- * Next, compare $A(2)$ & 8, no match occurs.
- * compare $A(3)$ & $A(4)$ with 8, no match occurs.
- * Up to now 4 comparisons take place. Now compare $A(5)$ & 8 (i.e $8=8$), So match occurs. (Search element found in the array).
- * Therefore no. of comparisons are 5. It is observed that Search takes average no. of comparisons. So it comes under average case.

Note: If there are n elements, then we require $n/2$ comparisons

$\therefore$ Time complexity is $O(\frac{n}{2}) = \underline{O(n)}$ (we can neglect constant).

WORST CASE:

- * If we want to search an element 13, whether it is present in the array (or) not.
- * first $A(1)$ is compared with 13 (i.e $2=13$), No match occurs. Continue this process until either element is found or the list is exhausted.
- * The element is found at 13th comparison. So No. of comparisons are 13.

* Therefore Search takes maximum number of comparisons. So it comes under worst case

(16)

∴ Time complexity is $O(n)$.

ASYMPTOTIC NOTATION

- * Asymptotic Notation is a way of comparing function that ignores factors and small input sizes.
- * Here Asymptotic means study of function of parameter 'n' and 'n' becomes larger and larger without bound.
- * Here we are concerned about how the running time of an algorithm increases with the size of the input.
- * These three notations are used to calculate the running time complexity of an algorithm.

(1) Big-oh Notation (O)

(2) Omega Notation (Ω)

(3) Theta Notation (Θ) (or) ($\otimes$)

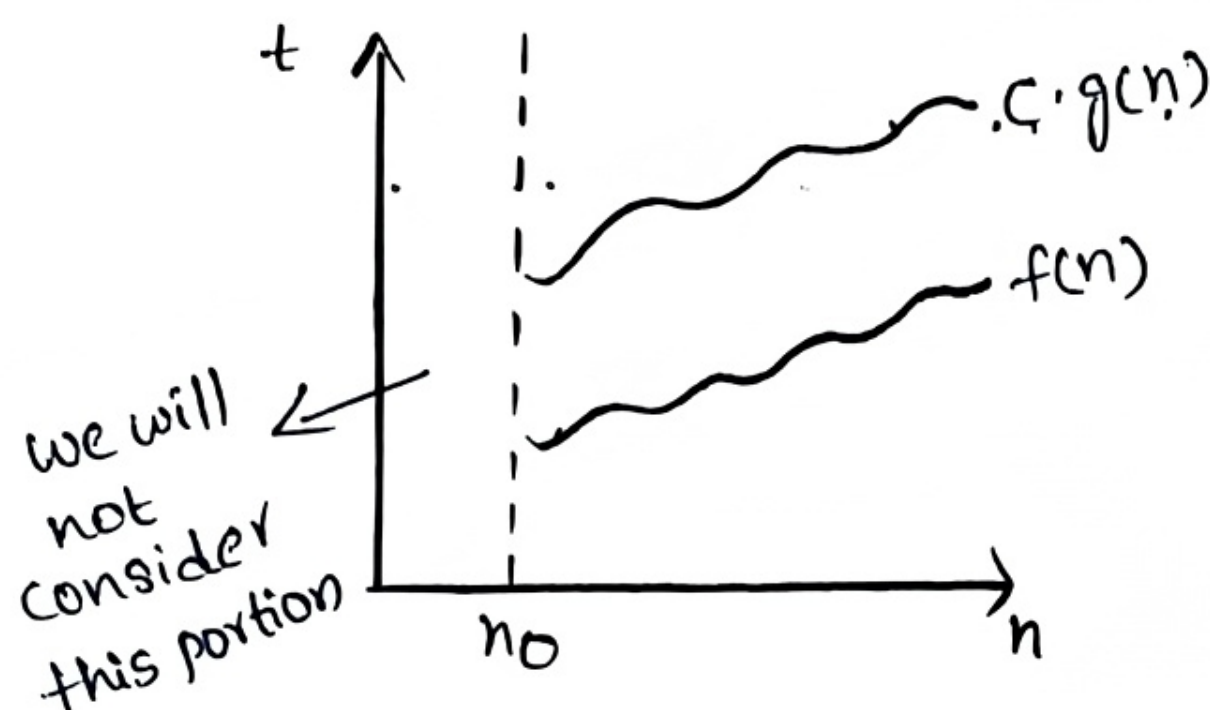
(1) Big-oh Notation:

- * Big-oh is the formal method of expressing the upper bound of an algorithm's running time.
- * It is the measure of the longest amount of time.
- * The upper bound of $f(n)$ indicates that the function $f(n)$ will be the worst case.

fig: Asymptotic upper bound is defined below.

Definition: The function $f(n) = O(g(n))$ such that $\exists$ two positive constants 'c' & 'n₀' with the constraint that,

$$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$



Big-oh Notation $f(n) \in O(g(n))$

Examples:

Eg 1) Give $f(n) = 3n + 2$, then p.T $f(n) = O(n)$.

Sol: $f(n) = 3n + 2$

Here $f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$
...

$$g(n) = n$$

Here always $n_0 = 1 \quad n \geq 1$

$$3n + 2 \leq c \cdot n$$

$$3(1) + 2 \leq c(1)$$

$$3(1) + 2 \leq c \quad \therefore c = 5$$

$$5 \leq 5$$

$$\therefore \boxed{f(n) = O(n)}$$

Eg 2) Given $f(n) = 20n^3 - 3$, then prove that $f(n) = O(n^3)$

Sol: $f(n) = 20n^3 - 3$.

$$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

$$f(n) = 20n^3 - 3 \quad g(n) = n^3 \quad c = 20 \quad n = 1$$

$$20n^3 - 3 \leq 20 \cdot n^3$$

$$20 - 3 \leq 20$$

$$18 \leq 20$$

$$\therefore f(n) = O(g(n))$$

$$f(n) = O(n^3)$$

eg 3: Given $f(n) = 10n^2 + 4n + 3$, then p.T $f(n) = O(n^2)$

Sol $f(n) = 10n^2 + 4n + 3$

$$g(n) = n^2$$

$$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

$$10n^2 + 4n + 3 \leq c \cdot n^2$$

$$c = 17 \quad n = 1$$

$$10 + 4 + 3 \leq 17 \cdot 1$$

$$17 \leq 17$$

$$f(n) = O(g(n))$$

$$f(n) = O(n^2)$$

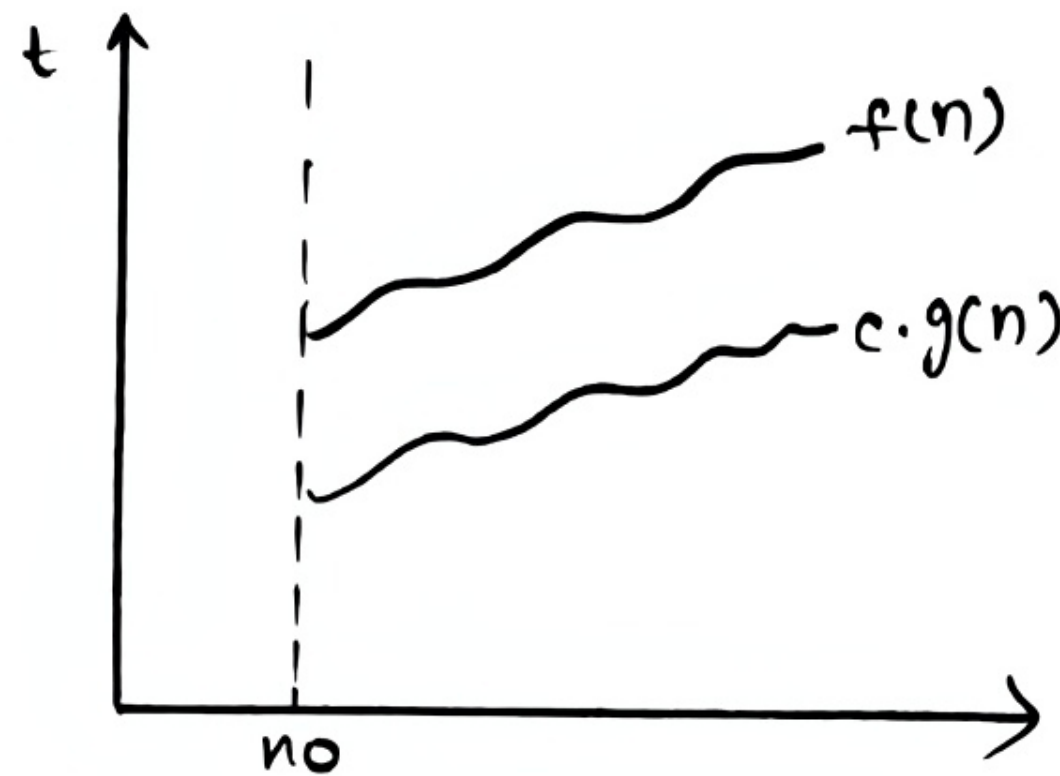
2) Omega Notation (Ω)

* This notation is used to find the lower bound behaviour of $f(n)$.

* It is represented mathematically by notation Ω .

Definition: $f(n) = \Omega(g(n))$ iff there exists two positive integer constants 'c' and 'no' with the constraint that,

$$f(n) \geq c \cdot g(n) \quad \forall n \geq n_0$$



Omega notation $f(n) \in \Omega(g(n))$

Fig: Asymptotic Lower bound.

Examples:

1) Given $f(n) = 3n + 2$ then P.T $f(n) = \Omega(n)$

Sol Here

$$f(n) = 3n + 2, \quad g(n) = n$$

$$f(n) \geq c \cdot g(n) \quad \forall n \geq n_0 \quad \therefore n_0 = 1 \text{ [always constant]}$$

$\therefore n$ values varies by function.

$$3n + 2 \geq c \cdot n$$

$$n = 1 \quad c = 5$$

$$3(1) + 2 \geq 5 \cdot (1) \Rightarrow 5 \geq 5$$

$$\therefore f(n) = \Omega(g(n))$$

$$\therefore \boxed{f(n) = \Omega(n)}$$

2) Given $f(n) = 50n^2 + 10n - 5$ then P.T $f(n) = \Omega(n^2)$.

Sol $f(n) = 50n^2 + 10n - 5$

(18)

We have to p.T $f(n) = \Omega(n^2)$

Here $g(n) = n^2$ $c = 55$ $n = 1$

$$f(n) \geq c \cdot g(n)$$

$$50n^2 + 10n - 5 \geq c \cdot n^2$$

$$50 + 10 - 5 \geq 55$$

$$55 \geq 55$$

$$\therefore f(n) = \Omega(g(n))$$

$$\therefore f(n) = \Omega(n^2)$$

3) Theta Notation (Θ)

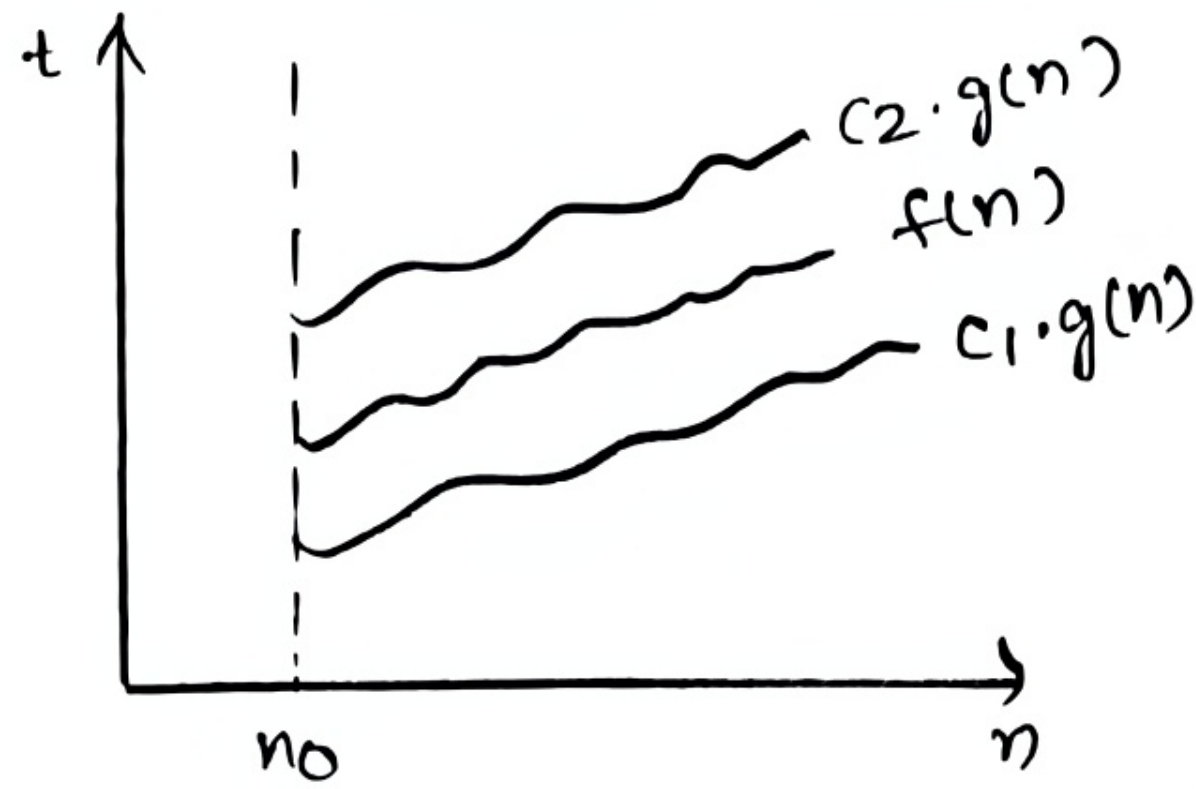
* For some functions the lower bound and upper bound may be same i.e. Big-oh and Omega will have the same function.

* For example, find the maximum (or) minimum element in the given array, then computing time for that min(max) element is $O(n)$ & $\Omega(n)$

* If the functions having the same time complexity for lower and upper bounds and this notation is used, it is called ' Θ ' notation.

Definition:- The function $f(n) = \Theta(g(n))$ if there exists three positive constants c_1, c_2 & n_0 with the constraint that,

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall n \geq n_0$$



Theta notation $f(n) \in \Theta(g(n))$

fig: Asymptotic Tight bound.

example:-

$$1) \text{ p.T } \frac{1}{2}n^2 - 3n = \Theta(n^2)$$

Sol $f(n) = \frac{1}{2}n^2 - 3n$

$$g(n) = n^2$$

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall n \geq n_0$$

$$c_1 \cdot n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 \cdot n^2$$

$$c_1 = -5/2 \quad c_2 = -5/2$$

$$n = 1$$

$$-5/2 \leq -5/2 \leq -5/2$$

$$\therefore f(n) = \Theta(g(n))$$

$$f(n) = \Theta(n^2)$$

TECHNIQUES FOR GRAPH

GRAPH TRAVERSAL TECHNIQUES:

* Traversing the graph means examining the nodes & vertices of the graph. There are two standard methods by using which, we can traverse the graphs.

* Graph traversal techniques are:-

1. Breadth first Search.
2. Depth first Search.

1. BREADTH FIRST SEARCH (BFS):